

VIVA GUIDE

B.Tech in Computer Science and Engineering

DIVISION OF COMPUTER SCIENCE AND ENGINEERING

SCHOOL OF ENGINEERING

COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY

Sanal Sivakumar

Project Details

Multi GNSS Dynamic Receiver Selection System

Technical viva preparation guide from fundamentals to implementation

Guide Details

Project Guide

SEPTEMBER 2026

How to Use This Guide

Read Sections 1 to 6 first if you are new to GNSS. They build the mental model needed to answer a teacher without memorising jargon. Sections 7 to 10 connect those concepts to the exact repository and its current limitations. Section 11 contains short answers to likely questions. Do not claim a result that this guide labels as a simulation result.

Thirty second project explanation

Our project compares two GNSS receiver readings at each update and selects the one with the larger reliability score $W = a*T + b*S + c*SNR + d*DOP$, where T is timing accuracy, S is satellite count, SNR is signal quality and DOP is inverted HDOP. The coefficients come from a small neural network that learns [a, b, c, d] from the current conditions, so the same transparent formula is always used while its weights adapt. A predefined fixed set is kept only as a bring-up fallback and a comparison baseline. We compare the learned model against the predefined fixed set and an optimized fixed set on held-out simulated sessions, and we integrated the adaptive weights into a Raspberry Pi decision engine with hysteresis and a fixed-weight fallback.

One sentence answer for the main contribution

We built a receiver-selection system whose reliability-equation coefficients are learned context-dependently by a compact network, and integrated the adaptive weights into the live engine with hysteresis and a safe predefined-weight fallback.

1. GNSS From First Principles

1.1 What does GNSS mean

GNSS means Global Navigation Satellite System. It is the broad name for satellite navigation constellations such as GPS, Galileo, GLONASS and BeiDou. GPS is one constellation; a GNSS receiver may use more than one constellation.

1.2 What is a GNSS receiver actually measuring

A satellite broadcasts a known signal with timing information. The receiver compares the expected code timing with the received code timing. Multiplying the measured delay by the speed of light gives an approximate distance called a pseudorange. It is called pseudo because the receiver clock and other sources add error. With at least four satellites, the receiver estimates x , y , z and clock bias together.

1.3 Why four satellites

A 3D position has three unknown coordinates. The receiver clock offset is a fourth unknown because its clock is not perfectly synchronized with satellite time. Four independent pseudorange equations provide enough information to estimate all four unknowns. More than four satellites allow an overdetermined solution, which can improve robustness.

1.4 Main error sources

Source	Plain-language explanation	Effect
Satellite clock and orbit	Broadcast timing and orbital data are not perfect.	Bias in pseudorange.
Ionosphere and troposphere	Signal speed changes slightly while passing through the atmosphere.	Distance error that depends on path and conditions.
Multipath	The receiver sees reflected as well as direct signals.	Can cause metre-level bias and spikes.
Obstruction	Buildings or trees block satellites or weaken signals.	Fewer usable satellites and poorer geometry.
Receiver noise	Low-cost electronics and antenna limits add tracking noise.	Jitter in time and position.
Reference mismatch	A test reference may itself be wrong or not aligned to the antenna.	Can make measured model error misleading.

Table 1. Sources of GNSS position error.

2. Quality Indicators You Must Understand

2.1 Satellite count

Satellite count is a useful clue because more tracked satellites can provide redundancy. It is not a guarantee: ten strong, well spread satellites can be better than twenty weak or clustered ones. The normalized term uses satellites / 20 capped at 1, and the raw count also feeds the conditioning vector.

2.2 HDOP

HDOP stands for Horizontal Dilution of Precision. It describes how the satellite geometry amplifies measurement errors into horizontal position errors. Smaller HDOP is generally better. HDOP does not include every error source and is not equal to metre error. The score uses $1/\text{HDOP}$ capped at 1; the raw HDOP is also a conditioning feature.

2.3 C/N0

C/N0 is carrier-to-noise density ratio in dB-Hz. Higher values usually mean the receiver tracks the signal more comfortably above noise. It is more precise terminology than a generic SNR label in GNSS contexts. Our firmware averages the per-satellite C/N0 over one second and sends it in the frame. A production system should decide how to aggregate per-satellite values and log the method.

2.4 Time accuracy and position step

Time accuracy answers whether a reading is fresh by comparing the reported receiver time with the live UTC clock. A position can be valid but stale, which is dangerous during movement. Position step is the distance between the current and previous reported positions. A large step can be real motion, an outlier or a receiver jump, so it is useful only in combination with the other features.

2.5 HDOP and satellite count are related but not identical

They often move together because fewer visible satellites can worsen geometry. They are still different: two groups of satellites can have the same count but very different sky distribution and HDOP. A model can receive both, but it needs properly separated data and careful validation so it does not learn a fragile accidental correlation.

3. The Weighted Score and the Predefined Fallback

```
T = clip(1 - time_error/5, 0, 1)
S = clip(satellites/20, 0, 1)
SNR = clip(cnr/50, 0, 1)
DOP = clip(1/HDOP, 0, 1)
```

```
W = 0.25*T + 0.25*S + 0.30*SNR + 0.20*DOP    (fallback set)
select the receiver with the larger W
```

3.1 Why the predefined set is useful as a fallback

The formula uses understandable signals and produces one number per receiver. It is cheap to calculate and easy to explain. A predefined coefficient set is useful during bring-up to verify that acquisition and decision logic work before the learned weights are enabled, and as a safe fallback when the model is unavailable.

3.2 Why the predefined set is not the decision method

- The numbers 0.25, 0.25, 0.30 and 0.20 are convenient starting values, not values learned from labelled position-error observations.
- The weights never change, so the formula cannot react when the GNSS environment changes.
- The terms clip high satellite counts and good HDOP values, losing distinctions in those ranges.
- It can double-count quality evidence when inputs are correlated.
- A score has no direct physical unit. Without calibration, 0.8 does not mean 80 percent reliable.
- Invalid inputs must be explicitly rejected; otherwise a missing value may accidentally look favourable.

3.3 Difference between predefined and learned weights

Question	Predefined fallback	Learned weights
Where do weights come from	Human choice.	Optimisation on labelled training examples (or grid search for Model B).
Can weights change with context	No.	Yes: $[a,b,c,d] = f(X)$ changes with conditions.
What is fixed	The formula and the coefficients.	The formula; the coefficients change.
What can go wrong	Poor fixed formula or thresholds.	Bad or unrepresentative data, leakage, overfitting, or unsafe outputs without a fallback.

4. The Adaptive Weight Model

Trainable parameters: 1,108 (408 + 600 + 100). Stored normalization values: 32 (mean and std for 16 inputs). Output coefficients are the adaptive weights used

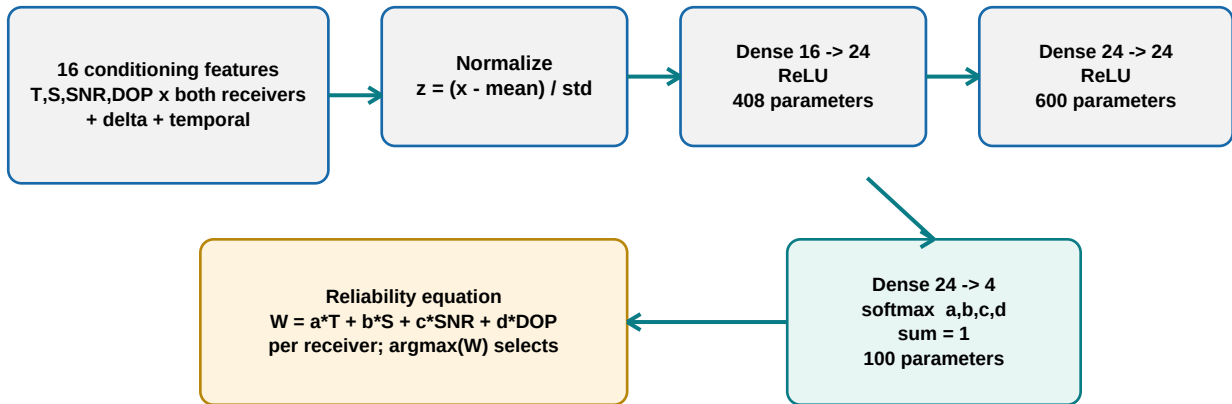


Fig 4.1 Learned weights: conditioning features produce softmax coefficients for the reliability equation.

4.1 Exact architecture and parameter count

Input X: 16 conditioning features
Normalize: $z = (x - \text{training_mean}) / \text{training_std}$
Hidden 1: $\text{ReLU}(z @ W1 + b1)$ 16 -> 24
Hidden 2: $\text{ReLU}(h1 @ W2 + b2)$ 24 -> 24
Output: $\text{softmax}(h2 @ W3 + b3)$ 24 -> 4 [a, b, c, d]

W1: 16 x 24 = 384 b1: 24
W2: 24 x 24 = 576 b2: 24
W3: 24 x 4 = 96 b3: 4
Total trainable parameters = 1,108

4.2 What does ReLU do

ReLU means Rectified Linear Unit. It applies $\max(0, \text{value})$. Negative hidden activations become zero while positive activations pass through. Stacking ReLU layers lets a small network learn nonlinear patterns, such as a case where HDOP matters differently when C/N0 is weak and a reading is stale.

4.3 Why softmax and why sum to one

Softmax converts the four output logits into non-negative values that sum to one. This makes [a,b,c,d] interpretable as a normalized allocation of importance across timing, satellites, signal and geometry. It is a modelling choice, not an empirical law; the network could in principle learn weights with different constraints.

4.4 Why normalize inputs

Features have very different scales. Without normalization, large-scale inputs can dominate optimisation just because of their units. The model stores the training mean and standard deviation and applies the same transformation at inference.

4.5 What the model does not do

- It does not calculate a new latitude or longitude.
- It does not fuse both receiver positions.
- It does not output a calibrated probability of reliability.
- It does not guarantee a maximum error.
- It never replaces the decision rule; it only provides the coefficients.

5. Data, Labels and Training

5.1 What is one training example

One training example is one epoch (one time step) where both receivers are available. Its input is the 16-dimension conditioning vector assembled from both receivers. Its targets for the loss are the two horizontal errors, in metres, between each receiver's reported position and the reference trajectory.

5.2 Why the reference position is not an input

If the reference position were given to the model as an input, it would leak the answer. In a real system, the reference position is not available while selecting a receiver. It is used only during training and evaluation to compute the errors in the loss.

5.3 The training objective

The network outputs weights, faces the reliability scores $W1$ and $W2$, and is trained so that soft selection expected error is small: $\text{loss} = P1 \cdot E1 + P2 \cdot E2$ with $P1 = \text{sigmoid}(\tau \cdot (W1 - W2))$. The temperature τ anneals from 1.98 to 1.55 so training starts smooth and ends close to the hard argmax deployed at runtime.

5.4 Controlled data design

The generator produces five conditions: open sky, partial obstruction, urban multipath, dropout and mixed. It includes quality changes, reported position noise, persistent bias, shared drift, occasional large errors, stale output intervals and invalid fixes. The goal is to test the pipeline under varied controlled cases. It is not a substitute for collecting real receiver logs with a surveyed reference.

5.5 Split, loss and early stopping

Term	Meaning in this project
Train split	80 full sessions used to adjust network parameters.
Validation split	20 separate sessions used to choose the best epoch, grid-search Model B and stop early.
Test split	20 separate sessions used once for final reported metrics.
Loss	Expected positioning error under soft selection, with temperature annealing.
Optimizer	Adam, learning rate 0.001, batch 512, patience 12, max 60 epochs.
Best checkpoint	Epoch 6 based on lowest validation expected error.

6. How Inference Chooses a Receiver

```
for each receiver: check eligibility, compute T, S, SNR, DOP
[a, b, c, d] = f(X)           # learned or fixed fallback
W_i = a*T_i + b*S_i + c*SNR_i + d*DOP_i

if |W0 - W1| < hysteresis threshold:
    keep the previously selected receiver
else:
    select argmax(W) among available receivers (else none)
```

6.1 Eligibility rules

Input	Accepted condition	Reason
Fix	Valid latitude and longitude	A receiver without a fix cannot provide a position.
Satellite count	At least 4	A normal 3D fix needs enough satellites for position plus clock bias.
HDOP	In (0, 99.99)	Zero, negative or the invalid default is rejected.
Age	0 to 2 seconds	Old samples should not win a real-time decision.
C/N0	Positive (when present)	For ML use both receivers must report a positive 1 s average.

6.2 Hysteresis and why it matters

Suppose receiver A scores 0.70 and receiver B scores 0.69 at one second, then their order reverses the next second. Immediate switching can oscillate, changing the selected source several times per minute. Hysteresis keeps the previous selection unless the difference crosses a threshold. In this study threshold 0.05 was the best balance, cutting switches by about 42 percent with unchanged or slightly better accuracy. The correct threshold must be confirmed on real data.

6.3 Fallback behaviour

If the model fails to load or returns abnormal weights, the engine falls back to the predefined fixed coefficients 0.25, 0.25, 0.30, 0.20 (Model A). The fallback keeps the live service functional and is also used for the 7-field frames that lack a C/N0 value.

7. Results You Can Explain Precisely

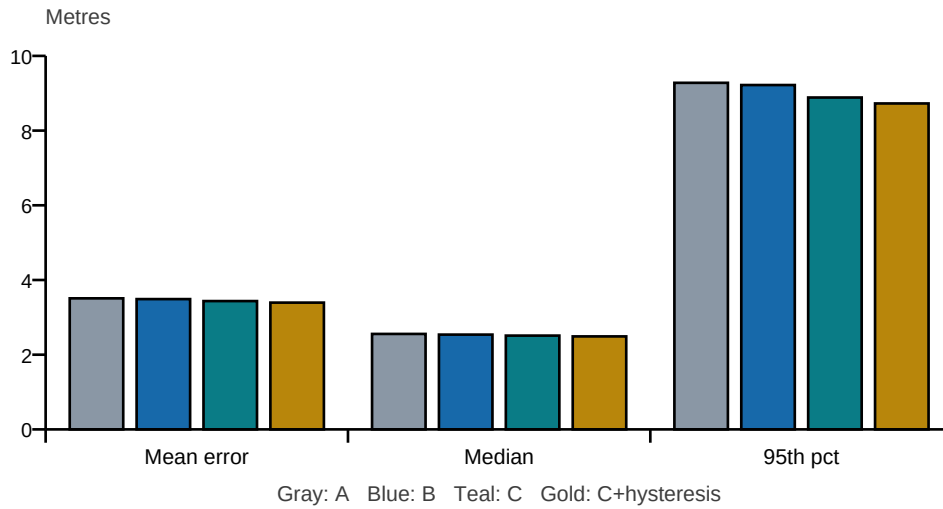


Fig 7.1 Overall held-out result from the controlled experiment.

Selector	Accuracy vs oracle	Mean m	Median m	95th m	Switches
Model A - predefined fixed	61.3%	3.509	2.556	9.279	1,788
Model B - optimized fixed	61.6%	3.488	2.538	9.220	1,768
Model C - dynamic ML	62.6%	3.435	2.511	8.886	1,767
Model C + hysteresis (0.05)	64.0%	3.394	2.489	8.727	1,018

7.1 A correct interpretation

On the held-out sessions, Model C achieved 3.435 m mean selected error versus 3.509 m for the predefined fixed fallback and 3.488 m for the optimized fixed set. With the 0.05 deadband the mean falls to 3.394 m. The gains are real but modest; the bigger practical win is the switching reduction. It does not prove that the model will achieve these values on a road, farm or city street.

7.2 Learned weights are interpretable

Overall the model allocates about 58 percent to DOP, but in open-sky epochs the DOP weight reaches 1.00 while in urban multipath the timing and satellite weights dominate ($\alpha + \beta \sim 0.96$). This is the key evidence that the network learned context-dependent weighting rather than a single fixed allocation.

7.3 The flip-oracle is not a competitor

The oracle compares the actual reference errors after the fact and represents the maximum agreement any selector could achieve. It is shown only to calibrate how much room for improvement remains in choosing between these two outputs. It must never be described as an implementable method.

8. Exact Code Flow and Current Limits

8.1 ESP32 firmware

The sketch creates two TinyGPSPlus parsers and two ESP32 hardware serial ports (UART1 RX1 18 / TX1 5, UART2 RX2 26 / TX2 25). Each loop feeds bytes into the parsers and a small C/N0 tracker that consumes GSV sentences. When a location updates, it emits GPS or GNSS records with time, latitude, longitude, satellite count, HDOP and the 1 s average C/N0.

8.2 Raspberry Pi decision engine and server

gnss_engine.py parses the frame, builds receiver records with the raw channels (satellites, HDOP, C/N0, age, step, time error, receiver id), computes the reliability terms and the 16-dimension conditioning vector, obtains weights from DynamicWeights with a fixed-weight fallback, applies the hysteresis deadband and logs every decision to decisions.jsonl. server.py runs the engine in a background thread and serves /location, with a --simulate mode that feeds synthetic frames when no hardware is attached.

8.3 Offline model files

File	Purpose
ml/generate.py	Creates controlled sessions, reference trajectory, observations, features and errors.
ml/features.py	Builds the 16-dimension conditioning features and the reliability terms.
ml/baselines.py	Model A fixed weights and the Model B grid search over the simplex.
ml/train.py	Splits sessions, trains Model C, exports weights and metrics.
ml/predict.py	Portable NumPy inference: DynamicWeights, hysteresis and selection.
ml/metrics.py	Evaluation, switching statistics and hysteresis application.
ml/evaluate.py	Reproduces the full table, hysteresis sweep and ablation.
ml/artifacts/model.npz	Saved weights, biases and 32 normalization values.
ml/tests/test_pipeline.py	Checks determinism, features, training, inference and session isolation.
raspberrypi/gnss_engine.py	Live engine that loads model.npz and applies weights with fallback.
raspberrypi/tests/test_engine.py	Live engine tests: parsing, fallback, hysteresis, logging.

8.4 Hard question: is this live machine learning on the Raspberry Pi

Yes, in prototype form. The exported model.npz is loaded at runtime by gnss_engine.py and the engine is unit-tested. The remaining honesty point is that 'live' here means the software pipeline is connected end to end; it has not been validated against real receiver logs with a trusted reference. Real-field validation and retraining are the next step.

9. Practical Defence Checklist

9.1 Statements you can safely make

- The architecture supports two receiver inputs, embedded acquisition with C/N0, and a live decision engine.
- The reliability equation $W = a*T + b*S + c*SNR + d*DOP$ is the fixed decision rule; the coefficients are learned and context-dependent.
- The model is compact: 1,108 trainable parameters and 32 stored normalization values.
- Invalid, stale or unsupported inputs are rejected by explicit eligibility rules.
- Whole sessions were kept separate between training, validation and testing.
- The learned weights are interpretable and shift with GNSS conditions (DOP-dominated in open sky, timing and satellite-dominated in urban multipath).
- The live engine is integrated with hysteresis and a fixed-weight fallback and is covered by unit tests.
- In the recorded controlled study the learned selector improved over the fixed baselines, and hysteresis cut switching by about 43 percent.

9.2 Statements you must not make

- Do not say a score of 0.8 proves 80 percent reliability.
- Do not present the recorded metres as RTK, surveyed-point or field-test accuracy.
- Do not say the engine has been validated with real receiver logs.
- Do not say the model corrects GNSS coordinates or fuses two positions.
- Do not call the flip-oracle a deployable selector.
- Do not claim the learned weights are causal; they are aggregated observations from the controlled study.

9.3 A good answer when challenged about data

This stage is a controlled model-development experiment. It verifies the training pipeline, session split, invalid-input handling, model export, fallback behaviour and comparison against fixed baselines. For deployment-grade claims, we would collect synchronized readings and a trusted reference trajectory, retrain with held-out real routes, and report error, availability and switch behaviour by environment.

9.4 Suggested demonstration sequence

- 1 Show the architecture diagram and explain the flow from two receivers through ESP32 and Raspberry Pi.
- 2 Show one combined serial-line example and identify each field including the C/N0.
- 3 Show the reliability equation and the predefined Model A fallback coefficients.
- 4 Show the adaptive model: 16 features, 1,108 parameters, softmax weights feeding the equation.
- 5 Show the train, validation and test session split, then the held-out chart.
- 6 Show the eligibility checks, the hysteresis deadband and the fixed-weight fallback.
- 7 Run `server.py --simulate` and show /location changing as the simulated conditions change, plus the JSONL log.

8 End with the field-validation plan and the evidence boundary.

10. Possible Viva Questions and Answers

1. *What problem does your project solve?*

It selects the more reliable output from two GNSS receivers at each update. The aim is to reduce selected-position error when receiver quality changes due to geometry, blockage, multipath, signal strength or stale data.

2. *Why use two receivers instead of one?*

A single low-cost receiver can degrade unexpectedly. Two receivers provide alternative observations. The system does not create an ideal position from nothing; it chooses the receiver that looks more reliable at that moment.

3. *What is GNSS and how is it different from GPS?*

GNSS is the general term for satellite navigation systems. GPS is one constellation. A GNSS receiver may also use Galileo, GLONASS or BeiDou depending on its hardware and configuration.

4. *Why do you need at least four satellites?*

The receiver must solve three position coordinates and its own clock bias. Four unknowns require at least four independent pseudorange observations.

5. *What is HDOP?*

Horizontal Dilution of Precision measures the effect of satellite geometry on horizontal position uncertainty. Lower is generally better. It is not the same thing as actual horizontal error.

6. *What is C/N0?*

Carrier-to-noise density ratio in dB-Hz. It describes how strong and trackable a satellite signal is relative to noise. Our ESP32 averages the per-satellite GSV values over one second into the snr field.

7. *Why can a receiver with more satellites still be worse?*

Satellite count does not tell us geometry, multipath level, signal integrity or receiver-specific bias. A larger count can still be poor if satellites are clustered or reflected signals dominate.

8. *What is multipath?*

Multipath occurs when the receiver receives reflected copies of a satellite signal in addition to the direct path. The reflected path is longer, so the timing estimate can be biased and the position can jump.

9. *State the reliability equation.*

$W = a \cdot T + b \cdot S + c \cdot \text{SNR} + d \cdot \text{DOP}$, where T is timing accuracy, S is the satellite-count term, SNR is the signal-quality term and DOP is inverted HDOP. We select the receiver with larger W .

10. *What does each term in the equation mean?*

$T = \text{clip}(1 - \text{time_err}/5, 0, 1)$, $S = \text{clip}(\text{sat}/20, 0, 1)$, $\text{SNR} = \text{clip}(\text{cn0}/50, 0, 1)$, $\text{DOP} = \text{clip}(1/\text{hdop}, 0, 1)$. Each is normalized to 0 to 1.

11. *What are the predefined fallback weights?*

$a=0.25$, $b=0.25$, $c=0.30$, $d=0.20$. This is Model A; the live engine keeps them as an emergency fallback and they serve as a comparison baseline in the study.

12. Why are predefined weights a weak primary method?

They were selected by design choice rather than fitted to labelled receiver errors. There was no experiment showing signal quality deserves exactly 30 percent and geometry exactly 20 percent, and they never change with conditions.

13. Why is capping satellite count or HDOP a concern?

It treats all counts above 20 (or all HDOP values below 1) as equal for that term. The system loses information that may still matter.

14. What is Model B?

A fixed coefficient set found by grid search over the simplex at step 0.05 - 1,771 candidates evaluated on validation epochs. It selected $a=0.10$, $b=0.20$, $c=0.00$, $d=0.70$.

15. What does the learned model actually output?

It outputs the four weights [a, b, c, d]. It does not predict a position or an error directly; the coefficients are plugged into the same reliability equation as before.

16. Why softmax the output?

Softmax forces the four coefficients to be non-negative and sum to one, so they read as a normalized allocation of importance. It also guarantees bounded weights at inference.

17. What are the model inputs?

A 16-dimension conditioning vector: for both receivers the four reliability-relevant values (timing error, satellites, C/N0, HDOP), plus four comparative deltas and four temporal statistics (satellite rate and a 5-sample C/N0 moving average per receiver).

18. What is normalization?

For each feature, subtract the training mean and divide by the training standard deviation. It puts different units on comparable scales for stable optimisation.

19. How many parameters does the model have?

1,108 trainable: 408 in the first dense layer (16 to 24), 600 in the second (24 to 24), 100 in the output layer (24 to 4). It also stores 32 non-trainable normalization values.

20. Why did you choose such a small network?

The input is only 16 values and the system targets lightweight inference on the Raspberry Pi. A small model is easier to debug, inspect and validate.

21. What is the training loss?

The expected positioning error under soft selection: $\text{loss} = P1 \cdot E1 + P2 \cdot E2$, where $P1 = \text{sigmoid}(\tau \cdot (W1 - W2))$ and $E1$, $E2$ are the actual generated errors of the two receivers.

22. What is the temperature tau?

A sharpness parameter: low tau makes selection soft and differentiable; high tau approaches the hard argmax. It is annealed from about 1.98 to 0.55 across training so the final behaviour matches deployment.

23. What is data leakage?

Leakage happens when information from the evaluation set influences training or model selection. We prevent a common time-series form by separating whole sessions, not random neighbouring rows.

24. *Why split by session rather than random rows?*

Adjacent seconds in one journey share conditions and are highly correlated. Random row splitting can put almost identical points into train and test.

25. *What is validation data used for?*

Validation data selects the training settings, the best epoch and the Model B grid search. The final results come from separately held-out test sessions.

26. *What is early stopping?*

It stops training when validation performance no longer improves for a chosen number of epochs (patience 12 here). The best checkpoint was epoch 6.

27. *What is flip accuracy vs the oracle?*

The fraction of available epochs where the selector picks the receiver that actually had the lower error. The oracle is computed after the fact and is an unattainable upper bound, used only to calibrate.

28. *What are the held-out results?*

Model A mean error 3.509 m at 61.3% agreement; Model B 3.488 m; Model C 3.435 m; Model C with 0.05 hysteresis 3.394 m at 64.0%.

29. *How large is the improvement?*

Real but modest: Model C is about two percent better in mean error than Model A in this controlled study. The larger practical gain is stability - the hysteresis deadband cuts switches by about 43 percent.

30. *Can you claim this accuracy in the real world?*

No. The recorded figures are from a controlled simulation study. Real deployment requires synchronized real receiver logs and a trusted reference system, then retraining and held-out field evaluation.

31. *What happens if an input is negative or missing?*

The eligibility checks reject invalid or stale receivers, so they cannot win selection. If neither receiver passes, the engine reports no selection rather than a fabricated location.

32. *What happens when there is no GNSS fix?*

That receiver is marked unavailable. The engine uses the other eligible receiver if available, otherwise it reports no selection.

33. *Why do you need hysteresis?*

Without it, very close scores can cause the selected source to flip every update, producing a noisy output track and more position jumps. Hysteresis keeps the previous selection until the difference crosses a threshold.

34. *What hysteresis threshold is used live?*

0.05 by default, matching the best value in the simulation sweep. It is configurable and should be re-tuned on real data.

35. *Why is there a fixed-weight fallback?*

So the live system never depends on a single model file. If the model fails to load or returns abnormal weights, the engine uses the predefined fixed coefficients and keeps working.

36. Does your system fuse the positions?

No. It is a selector: it picks one receiver position. Sensor fusion would combine measurements using another estimator such as a Kalman filter, which is future work.

37. What is the role of ESP32?

The ESP32 acquires two GNSS UART streams, parses them with TinyGPSPlus, tracks C/N0 from GSV sentences and sends a compact combined message over serial.

38. What serial format does the ESP32 use?

Two receiver records separated by a vertical bar: GPS,time,lat,lon,sat,hdop,snr | GNSS,time,lat,lon,sat,hdop,snr. Latitude/longitude are NO_FIX when there is no fix.

39. Is the model connected to the live server?

Yes. gnss_engine.py loads model.npz at runtime, applies the weights with hysteresis and fallback, and logs every decision. The simulated frame mode lets you demo it without hardware.

40. How was the engine tested?

With unit tests (raspberrypi/tests/test_engine.py) covering frame parsing, availability, ML versus fallback selection, the hysteresis deadband and JSONL logging, plus an end-to-end simulated run against /location.

41. Which features are computed live?

The engine computes satellites, HDOP, C/N0, age, step and time error per receiver from the frame and its own state, forming the same 16-dimension conditioning vector used in training.

42. Why not just use the receiver's own accuracy field?

Receiver-reported accuracy can be useful, but its availability and calibration depend on the chipset. The project explores selection using a feature vector that can be logged, tested and extended.

43. What is RTK and why is it useful?

Real-Time Kinematic positioning uses carrier-phase corrections from a base or network reference to obtain much more accurate relative positions under suitable conditions. It can provide a strong reference for evaluating ordinary GNSS receivers, but it must itself be checked for solution quality and antenna alignment.

44. What is the difference between a surveyed stationary point and RTK reference?

A surveyed stationary point is a known fixed coordinate used while the antenna stays still. RTK provides a moving reference trajectory, useful for dynamic testing. Both need correct antenna reference point handling.

45. What real data would you collect next?

Synchronized raw observations and reported features from both devices, a trustworthy timestamp, reference coordinates, antenna offsets, route labels, fix status and quality flags; the existing JSONL log already records decisions and weights.

46. How would you evaluate a real deployment?

Split by complete real routes or days, train on some environments and test on unseen ones. Report mean, median, 95th percentile and worst tail behaviour, availability, switch rate and performance by environment.

47. How do you avoid overfitting?

Session-level held-out validation and test partitions, a small network, early stopping, and finally real routes that were not used for model development.

48. *Why not use a larger deep network?*

More parameters are not automatically better. The conditioning input is modest and the deployment needs simple inference. Architecture choice should be justified by held-out real-data comparisons.

49. *What are limitations of the present study?*

All results come from controlled generated trajectories, not field data. C/N0 is a single average without per-satellite aggregation; the same softmax-constrained weight form is a modelling choice; and the engine has not been validated against real receiver logs with a trusted reference.

50. *What is your future work?*

Collect synchronized reference-labelled field data, retrain by real receiver and environment, re-tune hysteresis, extend per-satellite C/N0 telemetry, and consider calibrated uncertainty or sensor fusion as separate extensions.

11 Last Minute Revision Sheet

Fact	Answer
Project goal	Choose the receiver with the larger reliability score and learn the score's weights adaptively.
Decision rule	$W = a*T + b*S + c*SNR + d*DOP$; $\text{argmax}(W)$ among available receivers.
Predefined fallback	Predefined coefficients (0.25/0.25/0.30/0.20) that never adapt; used only as a fallback and baseline.
Conditioning input	16 features from both receivers: reliability indicators, deltas and temporal statistics.
Architecture	16 -> 24 ReLU -> 24 ReLU -> 4 softmax [a,b,c,d].
Trainable parameters	1,108
Loss	Expected error under soft selection with temperature annealing.
Data split	80 train / 20 validation / 20 test whole sessions (stratified per scenario).
Best epoch	6
Test results	A 3.509 m, B 3.488 m, C 3.435 m, C+hyst 3.394 m.
Learned behaviour	DOP dominates in open sky; timing and satellite terms dominate in urban multipath.
Live status	Engine integrated and tested; field validation with real reference data remains.
Main evidence boundary	Controlled simulation result, not RTK or field accuracy.

Remember

A strong viva answer gives the mechanism, the reason for the design decision, and the boundary of the evidence. For example: HDOP measures geometry, so we keep it in the DOP term; the learned network re-weights the terms because conditions change; the model is integrated and tested, but only real labelled field data can establish performance for a real deployment.

Glossary

Term	Meaning
GNSS	Global Navigation Satellite System, a general term for satellite navigation constellations.
Pseudorange	Distance-like measurement from signal travel time that includes clock and other errors.
HDOP	Horizontal Dilution of Precision, a satellite-geometry indicator.
C/N0	Carrier-to-noise density ratio in dB-Hz, a GNSS signal-quality indicator.
Multipath	Reflected satellite signals that add timing and position bias.
RTK	Real-Time Kinematic carrier-phase correction technique for high-accuracy positioning.
Feature	Input value available to the model at decision time.
Hysteresis	A rule that prevents rapid switching when two scores are nearly equal.
Softmax	A transform that maps values to non-negative numbers summing to one.